Technical Neural Networks Assignment Sheet 3

Names: Mohamadhosein Sabaghi (50547423)

Assignment 12

$$ {}^{p}E^{*}(w_{ij}) = \sum_{m=0}^{M} ({}^{p}\hat{y}_m - {}^{p}y_m)^4 $$

Output layer. Only the term m depends on y_m (the teacher $\hat{y}_m$ is constant), so

$$ \frac{\partial E^*}{\partial y_m} = 4(\hat{y}_m - y_m)^3 \cdot (-1) = -4(\hat{y}_m - y_m)^3 $$

With $y_m = f(\text{net}_m)$ and $\text{net}_m = \sum_h w_{hm}\,\text{out}_h$:

$$ \frac{\partial E^*}{\partial \text{net}_m} = \frac{\partial E^*}{\partial y_m} \cdot f'(\text{net}_m) = -4(\hat{y}_m - y_m)^3 f'(\text{net}_m) $$

As in the lecture, define $\delta^*_m := -\partial E^*/\partial \text{net}_m$:

$$ \boxed{\delta^*_m = 4(\hat{y}_m - y_m)^3\, f'(\text{net}_m)} \qquad \text{(output layer)} $$

Hidden layer. A hidden neuron h influences E^* only through the net_m of every output neuron m it feeds into, $\text{net}_m = \sum_h w_{hm}\,\text{out}_h$:

$$ \frac{\partial E^*}{\partial \text{out}_h} = \sum_m \frac{\partial E^*}{\partial \text{net}_m} \cdot \frac{\partial \text{net}_m}{\partial \text{out}_h} = \sum_m (-\delta^*_m)\, w_{hm} = -\sum_m \delta^*_m w_{hm} $$

With $\text{out}_h = f(\text{net}_h)$:

$$ \frac{\partial E^*}{\partial \text{net}_h} = \frac{\partial E^*}{\partial \text{out}_h} \cdot f'(\text{net}_h) = -f'(\text{net}_h) \sum_m \delta^*_m w_{hm} $$

$$ \boxed{\delta^*_h = -\frac{\partial E^*}{\partial \text{net}_h} = f'(\text{net}_h) \sum_m \delta^*_m w_{hm}} \qquad \text{(hidden layer)} $$

Weight update. Exactly the usual delta-rule form, for any layer:

$$ {}^{p}\Delta w_{ij} = \eta\, {}^{p}\delta^*_j\, {}^{p}\text{out}_i $$

i.e. first compute δ^*_m for every output neuron, then propagate backward layer by layer using $\delta^*_h = f'(\text{net}_h)\sum_m \delta^*_m w_{hm}$, exactly as in standard Backpropagation of Error – only the output-layer δ^* changes (power 3 instead of 1, factor 4 instead of 1); everything downstream of it is identical.

Assignment 13

3-layer N-H-1 MLP, tanh in hidden and output layer, $\eta = 0.1$, $\hat{y}_{m=1} = 0.2$, $y_{m=1} = 0.8$, $w_{hm} = 2.5$, $o_h = 0.3$, $x_n = 15$.

Step 1 – output-layer δ_m. For tanh, $f'(\text{net}_m) = 1 - \tanh^2(\text{net}_m) = 1 - y_m^2$ (assignment 11b), so

$$\delta_m = (\hat{y}_m - y_m)\,(1 - y_m^2) = (0.2 - 0.8)(1 - 0.8^2) = (-0.6)(1 - 0.64) = (-0.6)(0.36) = -0.216$$

Step 2 – hidden-layer δ_h. Single output neuron, so the sum over m has one term:

$$\delta_h = f'(\text{net}_h) \cdot \delta_m\, w_{hm} = (1 - o_h^2) \cdot \delta_m\, w_{hm}$$

$$1 - o_h^2 = 1 - 0.3^2 = 1 - 0.09 = 0.91$$

$$\delta_h = 0.91 \cdot (-0.216) \cdot 2.5 = 0.91 \cdot (-0.54) = -0.4914$$

Step 3 – weight change.

$$\Delta w_{nh} = \eta\, \delta_h\, x_n = 0.1 \cdot (-0.4914) \cdot 15 = 0.1 \cdot (-7.371)$$

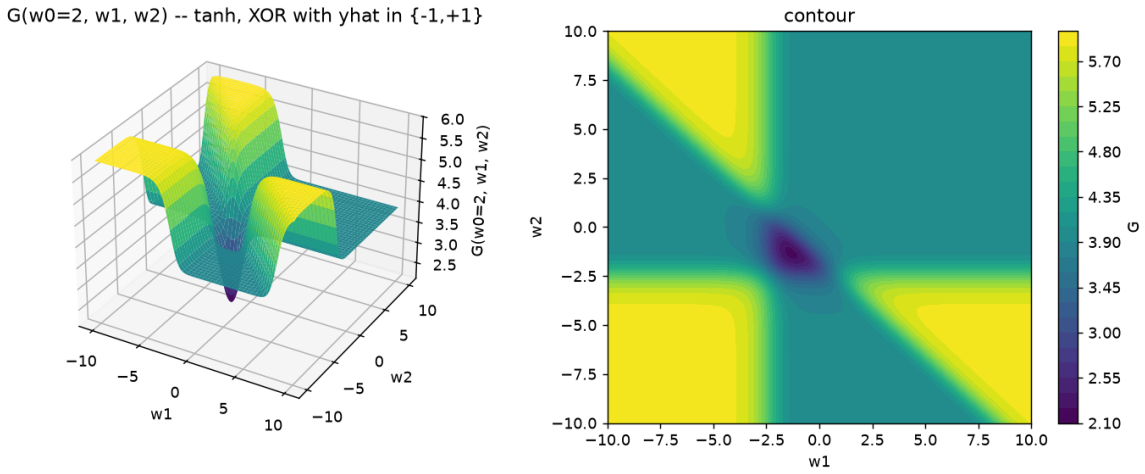
$$\boxed{\Delta w_{nh} = -0.7371}$$

Assignment 14

Network: $y = \tanh(w_0 + w_1 x_1 + w_2 x_2)$, XOR patterns $(x_1, x_2; \hat{y}) \in \{(0,0;-1), (0,1;+1), (1,0;+1), (1,1;-1)\}$, global error

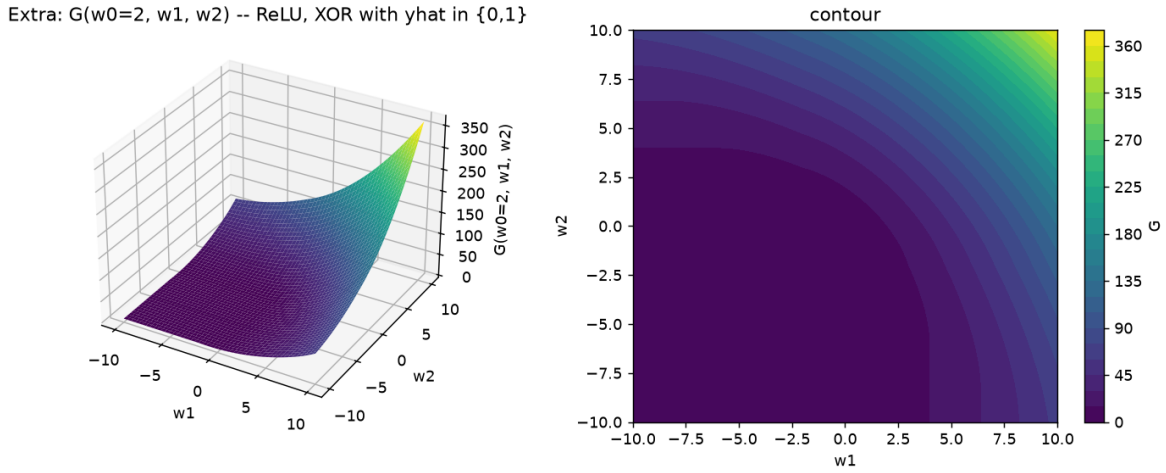
$$G(w_0, w_1, w_2) = \tfrac{1}{2} \sum_{p=1}^{4} (\hat{y}_p - \tanh(w_0 + w_1 x_{1,p} + w_2 x_{2,p}))^2$$

Computed on a grid over $-10 \le w_1, w_2 \le 10$ with $w_0 = 2.0$ fixed (full code in `error_surface.py`):



The surface has a single basin (minimum $G \approx 2.19$ around $w_1 \approx w_2 \approx -1.36$) that never reaches 0: with w_0 fixed, y is still just a (squashed) *linear* function of x_1, x_2, so this is really the same situation as Assignment 9 – no choice of w_1, w_2 can realize XOR, the surface just shows how far off the best possible linear fit remains.

Extra task (no extra points). Same network, but desired values $\{0, 1\}$ and transfer function ReLU ($f(z) = \max(0, z)$):



The shape is completely different: ReLU is unbounded and piecewise linear, so G is flat (≈ 0) over a large region (where the net input stays negative and ReLU simply outputs 0, which happens to match several $\hat{y} = 0$ patterns), and then grows without bound (quadratically) as w_1, w_2 increase, instead of the single smooth, bounded basin seen with tanh.

Assignment 15

AI use: I used Claude (Anthropic) to work through this part, as the sheet requires. I gave it the assignment text directly and asked it to compare single-step and cumulative Backpropagation of Error. The answer was usable as-is, so I didn't need to re-prompt or discard anything.

Single-step (online/stochastic) learning. One pattern p is presented, its Δw_{ij} computed, and the weights are updated *immediately*, before the next pattern is shown. This is what we used in PA-A: pick a (random) pattern, forward, backward, update, repeat.

Cumulative (batch) learning. All P training patterns are presented first, and their weight changes ${}^p\Delta w_{ij}$ are accumulated, $\Delta w_{ij} = \sum_{p=1}^{P} {}^p\Delta w_{ij}$, without touching the weights; only after the full epoch is the sum applied once. This is exact gradient descent on the total error $G = \sum_p {}^pE$.

- **Single-step:** + many more updates per epoch, usually converges faster in wall-clock terms; the noise from updating on one pattern at a time can help escape shallow local minima; needs no extra memory to accumulate Δw; works for streaming data. – the update direction is noisy and zig-zags (it follows one pattern's gradient, not G's true gradient); the result depends on the order patterns are presented in.
- **Cumulative:** + follows the true gradient of G, so convergence is smooth and order-independent; easier to analyze with standard gradient-descent theory. – requires a full pass over all patterns before a single update, so it's slower per update and needs more memory to accumulate Δw; with no stochastic noise it is more likely to get stuck exactly in a local minimum or saddle point; not applicable to online/streaming settings where the full pattern set isn't available upfront.